But we'd really like to be able to run arbitrary computations over fixed-size thread pools. In order to do that, we'll need to pick a different representation of `Par`.

7.4.4 A fully non-blocking Par implementation using actors

In this section, we'll develop a fully non-blocking implementation of `Par` that works for fixed-size thread pools. Since this isn't essential to our overall goals of discussing various aspects of functional design, you may skip to the next section if you prefer. Otherwise, read on.

The essential problem with the current representation is that we can't get a value *out* of a `Future` without the current thread blocking on its `get` method. A representation of `Par` that doesn't leak resources this way has to be *non-blocking* in the sense that the implementations of `fork` and `map2` must never call a method that blocks the current thread like `Future.get`. Writing such an implementation correctly can be challenging. Fortunately we have our laws with which to test our implementation, and we only have to get it right *once*. After that, the users of our library can enjoy a composable and abstract API that does the right thing every time.

In the code that follows, you don't need to understand exactly what's going on with every part of it. We just want to show you, using real code, what a correct representation of `Par` that respects the laws might look like.

THE BASIC IDEA

How can we implement a non-blocking representation of `Par`? The idea is simple. Instead of turning a `Par` into a `java.util.concurrent.Future` that we can get a value *out* of (which requires blocking), we'll introduce our own version of `Future` with which we can *register a callback that will be invoked when the result is ready*. This is a slight shift in perspective:

The `apply` method is declared `private` to the `fpinscala.parallelism` package, which means that it can only be accessed by code within that package.

```
sealed trait Future[A] {
  private[parallelism] def apply(k: A => Unit): Unit
}
type Par[+A] = ExecutorService => Future[A]
```

`Par` looks the same, but we're using our new non-blocking `Future` instead of the one in `java.util.concurrent`.

Our `Par` type looks identical, except we're now using our new version of `Future`, which has a different API than the one in `java.util.concurrent`. Rather than calling `get` to obtain the result from our `Future`, our `Future` instead has an `apply` method that receives a function `k` that expects the result of type `A` and uses it to perform some effect. This kind of function is sometimes called a *continuation* or a *callback*.

The `apply` method is marked `private[parallelism]` so that we don't expose it to users of our library. Marking it `private[parallelism]` ensures that it can only be accessed from code within the `fpinscala.parallelism` package. This is so that our API remains pure and we can guarantee that our laws hold.

Using local side effects for a pure API

The `Future` type we defined here is rather imperative. An `A => Unit`? Such a function can only be useful for executing some side effect using the given `A`, as we certainly aren't using the returned result. Are we still doing functional programming in using a type like `Future`? *Yes*, but we're making use of a common technique of using side effects as an implementation detail for a purely functional API. We can get away with this because the side effects we use are *not observable* to code that uses `Par`. Note that `Future.apply` is protected and can't even be called by outside code.

As we go through the rest of our implementation of the non-blocking `Par`, you may want to convince yourself that the side effects employed can't be observed by outside code. The notion of local effects, observability, and subtleties of our definitions of purity and referential transparency are discussed in much more detail in chapter 14, but for now an informal understanding is fine.

With this representation of `Par`, let's look at how we might implement the `run` function first, which we'll change to just return an `A`. Since it goes from `Par[A]` to `A`, it will have to construct a continuation and pass it to the `Future` value's `apply` method.

Listing 7.6 Implementing `run` for `Par`

A mutable, thread-safe reference to use for storing the result. See the `java.util.concurrent.atomic` package for more information about these classes.

When we receive the value, sets the result and releases the latch.

Waits until the result becomes available and the latch is released.

```
def run[A](es: ExecutorService)(p: Par[A]): A = {
  val ref = new AtomicReference[A]
  val latch = new CountDownLatch(1)
  p(es) { a => ref.set(a); latch.countDown }
  latch.await
  ref.get
}
```

Once we've passed the latch, we know `ref` has been set, and we return its value.

A `java.util.concurrent.CountDownLatch` allows threads to wait until its `countDown` method is called a certain number of times. Here the `countDown` method will be called once when we've received the value of type `A` from `p`, and we want the `run` implementation to block until that happens.

It should be noted that `run` blocks the calling thread while waiting for the `latch`. It's not possible to write an implementation of `run` that doesn't block. Since it needs to return a value of type `A`, it has to wait for that value to become available before it can return. For this reason, we want users of our API to avoid calling `run` until they definitely want to wait for a result. We could even go so far as to remove `run` from our API altogether and expose the `apply` method on `Par` instead so that users can register asynchronous callbacks. That would certainly be a valid design choice, but we'll leave our API as it is for now.

Let's look at an example of actually creating a `Par`. The simplest one is `unit`:

```
def unit[A](a: A): Par[A] =
  es => new Future[A] {
    def apply(cb: A => Unit): Unit =
      cb(a)
  }
```

Simply passes the value to the continuation. Note that the `ExecutorService` isn't needed.

Since `unit` already has a value of type `A` available, all it needs to do is call the continuation `cb`, passing it this value. If that continuation is the one from our `run` implementation, for example, this will release the latch and make the result available immediately.

What about `fork`? This is where we introduce the actual parallelism:

```
def fork[A](a: => Par[A]): Par[A] =
  es => new Future[A] {
    def apply(cb: A => Unit): Unit =
      eval(es)(a(es)(cb))
  }

def eval(es: ExecutorService)(r: => Unit): Unit =
  es.submit(new Callable[Unit] { def call = r })
```

`eval` forks off evaluation of `a` and returns immediately. The callback will be invoked asynchronously on another thread.

A helper function to evaluate an action asynchronously using some `ExecutorService`.

When the `Future` returned by `fork` receives its continuation `cb`, it will fork off a task to evaluate the by-name argument `a`. Once the argument has been evaluated and called to produce a `Future[A]`, we register `cb` to be invoked when that `Future` has its resulting `A`.

What about `map2`? Recall the signature:

```
def map2[A,B,C](a: Par[A], b: Par[B])(f: (A,B) => C): Par[C]
```

Here, a non-blocking implementation is considerably trickier. Conceptually, we'd like `map2` to run both `Par` arguments in parallel. When both results have arrived, we want to invoke `f` and then pass the resulting `C` to the continuation. But there are several race conditions to worry about here, and a correct non-blocking implementation is difficult using only the low-level primitives of `java.util.concurrent`.

A BRIEF INTRODUCTION TO ACTORS

To implement `map2`, we'll use a non-blocking concurrency primitive called *actors*. An `Actor` is essentially a concurrent process that doesn't constantly occupy a thread. Instead, it only occupies a thread when it receives a *message*. Importantly, although multiple threads may be concurrently sending messages to an actor, the actor processes only one message at a time, queueing other messages for subsequent processing. This makes them useful as a concurrency primitive when writing tricky code that must be accessed by multiple threads, and which would otherwise be prone to race conditions or deadlocks.

It's best to illustrate this with an example. Many implementations of actors would suit our purposes just fine, including one in the Scala standard library (see

scala.actors.Actor), but in the interest of simplicity we'll use our own minimal actor implementation included with the chapter code in the file `Actor.scala`:

```
scala> import fpinscala.parallelism._

scala> val S = Executors.newFixedThreadPool(4)
S: java.util.concurrent.ExecutorService = ...

scala> val echoer = Actor[String](S) {
     |   msg => println (s"Got message: '$msg'")
     | }
echoer: fpinscala.parallelism.Actor[String] = ...
```

An actor uses an `ExecutorService` to process messages when they arrive, so we create one here.

This is a very simple actor that just echoes the `String` messages it receives. Note we supply `S`, an `ExecutorService` to use for processing messages.

Let's try out this `Actor`:

```
scala> echoer ! "hello"
Got message: 'hello'

scala>

scala> echoer ! "goodbye"
Got message: 'goodbye'

scala> echoer ! "You're just repeating everything I say, aren't you?"
Got message: 'You're just repeating everything I say, aren't you?'
```

Sends the `"hello"` message to the actor.

Note that `echoer` doesn't occupy a thread at this point, since it has no further messages to process.

Sends the `"goodbye"` message to the actor. The actor reacts by submitting a task to its `ExecutorService` to process that message.

It's not at all essential to understand the `Actor` *implementation*. A correct, efficient implementation is rather subtle, but if you're curious, see the `Actor.scala` file in the chapter code. The implementation is just under 100 lines of ordinary Scala code.[15]

IMPLEMENTING MAP2 VIA ACTORS

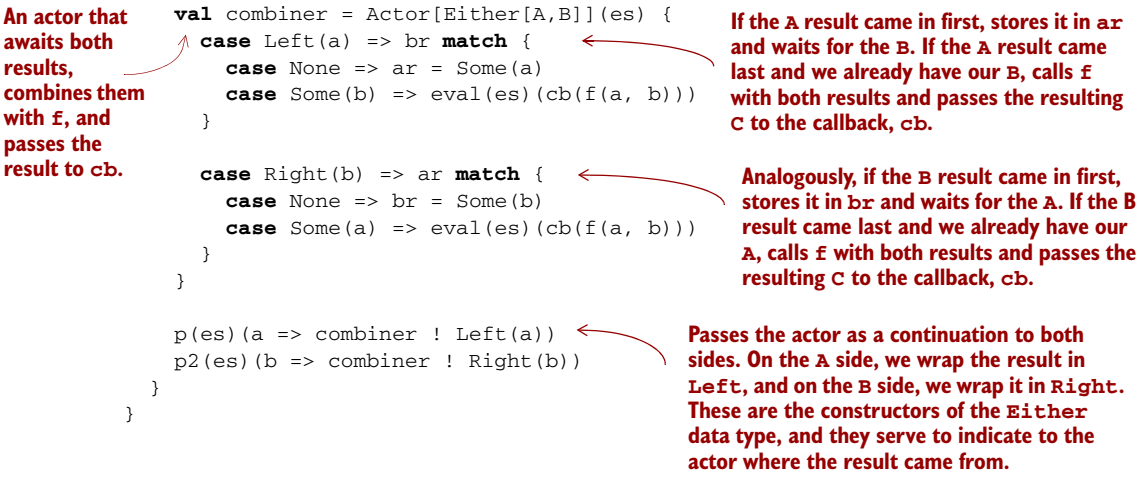
We can now implement `map2` using an `Actor` to collect the result from both arguments. The code is straightforward, and there are no race conditions to worry about, since we know that the `Actor` will only process one message at a time.

Listing 7.7 Implementing `map2` with `Actor`

```
def map2[A,B,C](p: Par[A], p2: Par[B])(f: (A,B) => C): Par[C] =
  es => new Future[C] {
    def apply(cb: C => Unit): Unit = {
      var ar: Option[A] = None
      var br: Option[B] = None
```

Two mutable `vars` are used to store the two results.

[15] The main trickiness in an actor implementation has to do with the fact that multiple threads may be messaging the actor simultaneously. The implementation needs to ensure that messages are processed only one at a time, and also that all messages sent to the actor will be processed eventually rather than queued indefinitely. Even so, the code ends up being short.




```
val combiner = Actor[Either[A,B]](es) {
  case Left(a) => br match {
    case None => ar = Some(a)
    case Some(b) => eval(es)(cb(f(a, b)))
  }

  case Right(b) => ar match {
    case None => br = Some(b)
    case Some(a) => eval(es)(cb(f(a, b)))
  }
}

p(es)(a => combiner ! Left(a))
p2(es)(b => combiner ! Right(b))
}
}
```

Given these implementations, we should now be able to run `Par` values of arbitrary complexity without having to worry about running out of threads, even if the actors only have access to a single JVM thread.

Let's try this out in the REPL:

```
scala> import java.util.concurrent.Executors

scala> val p = parMap(List.range(1, 100000))(math.sqrt(_))
p: ExecutorService => Future[List[Double]] = < function >

scala> val x = run(Executors.newFixedThreadPool(2))(p)
x: List[Double] = List(1.0, 1.4142135623730951, 1.7320508075688772,
2.0, 2.23606797749979, 2.449489742783178, 2.6457513110645907, 2.828
4271247461903, 3.0, 3.1622776601683795, 3.3166247903554, 3.46410...
```

That will call `fork` about 100,000 times, starting that many actors to combine these values two at a time. Thanks to our non-blocking `Actor` implementation, we don't need 100,000 JVM threads to do that in.

Fantastic. Our law of forking now holds for fixed-size thread pools.

EXERCISE 7.10

Hard: Our non-blocking representation doesn't currently handle errors at all. If at any point our computation throws an exception, the `run` implementation's `latch` never counts down and the exception is simply swallowed. Can you fix that?

Taking a step back, the purpose of this section hasn't necessarily been to figure out the best non-blocking implementation of `fork`, but more to show that laws are important. They give us another angle to consider when thinking about the design of a